THE FEDERAL POLYTECHNIC, MUBI

Department of Computer Science

COM 322
DATABASE DESIGN II
Lecture Note

MODULE I

TYPES OF DATA MODELS.

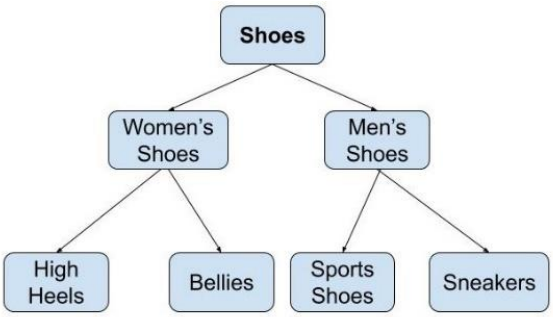
Data Model

Data Model defines the data elements and the relationships between the data elements. Data Models are used to show how data is stored, connected, accessed and updated in the database management system. It also gives us an idea that how the final system will look like after its complete implementation. Here, we use a set of symbols and text to represent the information so that members of the organization can communicate and understand it. Though there are many data models being used nowadays but the Relational model is the most widely used model. Apart from the Relational model, there are many other types of data models, some of these Data Models in DBMS are:

1. Hierarchical Model
2. Network Model
3. Entity-Relationship Model
4. Relational Model
5. Object-Oriented Data Model
6. Object-Relational Data Model

Hierarchical Model

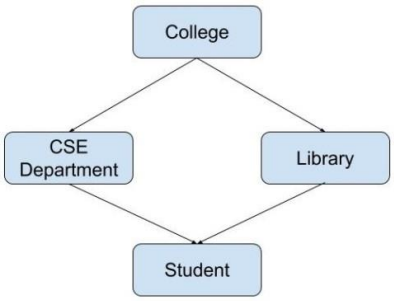
Hierarchical Model was the first DBMS model. This model organizes the data in the hierarchical tree structure. The hierarchy starts from the root which has root data and then it expands in the form of a tree adding child node to the parent node. This model easily represents some of the real-world relationships like food recipes, sitemap of a website etc. ***Example:*** We can represent the relationship between the shoes present on a shopping website in the following way:



Hierarchical Model

Network Model

This model is an extension of the hierarchical model. It was the most popular model before the relational model. This model is the same as the hierarchical model, the only difference is that a record can have more than one parent. It replaces the hierarchical tree with a graph. ***Example:*** In the example below, we can see that node student has two parents i.e. CSE Department and Library. This was earlier not possible in the hierarchical model.



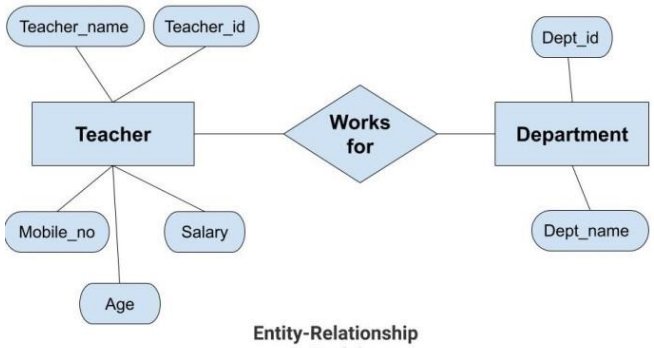
Network Model

Entity-Relationship Model

Entity-Relationship Model or simply ER Model is a high-level data model diagram. In this model, we represent the real-world problem in the pictorial form to make it easy for the stakeholders to understand. It is also very easy for the developers to understand the system by just looking at the ER diagram. We use the ER diagram as a visual tool to represent an ER Model. ER diagram has the following three components:

- ***Entities:*** Entity is a real-world thing. It can be a person, place, or even a concept. *Example:* Teachers, Students, Course, Building, Department, etc are some of the entities of a School Management System.
- ***Attributes:*** An entity contains a real-world property called attribute. This is the characteristics of that attribute. *Example:* The entity teacher has the property like teacher id, salary, age, etc.
- ***Relationship:*** Relationship tells how two attributes are related. *Example:* Teacher works for a department.

Example:



Entity-Relationship Model

In the above diagram, the entities are Teacher and Department. The attributes of ***Teacher*** entity are Teacher_Name, Teacher_id, Age, Salary, Mobile_Number. The attributes of entity ***Department*** entity are Dept_id, Dept_name. The two entities are connected using the relationship. Here, each teacher works for a department.

Relational Model

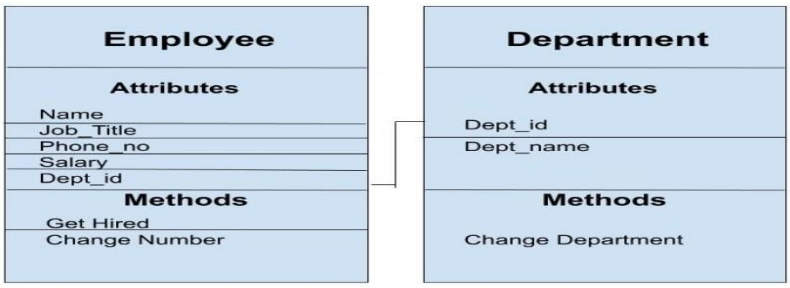
Relational Model is the most widely used model. In this model, the data is maintained in the form of a two-dimensional table. All the information is stored in the form of row and columns. The basic structure of a relational model is tables. So, the tables are also called *relations* in the relational model. ***Example:*** In this example, we have an Employee table.

Emp_id	Emp_name	Job_name	Salary	Mobile_no	Dep_id	Project_id
AfterA001	John	Engineer	100000	9111037890	2	99
AfterA002	Adam	Analyst	50000	9587569214	3	100
AfterA003	Kande	Manager	890000	7895212355	2	65

EMPLOYEE TABLE

Object-Oriented Data Model

The real-world problems are more closely represented through the object-oriented data model. In this model, both the data and relationship are present in a single structure known as an object. We can store audio, video, images, etc. in the database which was not possible in the relational model (although you can store audio and video in relational database, it is advised not to store in the relational database). In this model, two or more objects are connected through links. We use this link to relate one object to other objects. This can be understood by the example given below.



Object_Oriented_Model

CONCEPT OF OBJECT-ORIENTED LANGUAGES

It is necessary to understand some of the concepts used extensively in object-oriented programming. These include:

1. Objects
2. Classes
3. Data abstraction and encapsulation
4. Inheritance
5. Polymorphism

1 Objects

Objects are the basic run time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program has to handle. They may also represent user-defined data such as vectors, time and lists. Programming problem is analyzed in term of objects and the nature of communication
between them. Program objects should be chosen such that they match closely with the real-world objects. Objects take up space in the memory and have an associated address like a record in Pascal, or a structure in C.

2 Classes

We just mentioned that objects contain data, and code to manipulate that data. The entire set of data and code of an object can be made a user-defined data type with the help of class. In fact, objects are variables of the type class. Once a class has been defined, we can create any number of objects belonging to that class. Each object is associated with the data of type class with which they are created. A class is thus a collection of similar objects.

3 Data Abstraction and Encapsulation

The wrapping up of data and function into a single unit (called class) is known as encapsulation. Data and encapsulation are the most striking feature of a class. The data is not accessible to the outside world, and only those functions which are wrapped in the class can access it. These functions provide the interface between the object's data and the program. This insulation of the data from direct access by the program is called data hiding or information hiding.

4 Inheritance

Inheritance is the process by which objects of one class acquired the properties of objects of another classes. It supports the concept of hierarchical classification. For example, the bird, 'robin' is a part of class 'flying bird' which is again a part of the class 'bird'.

The principal behind this sort of division is that each derived class shares common characteristics with the class from which it is derived as illustrated. In OOP, the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. This is possible by deriving a new class from the existing one.

5 Polymorphism

Polymorphism is another important OOP concept. Polymorphism, a Greek term, means
the ability to take more than on form. An operation may exhibit different behavior is different instances. The behavior depends upon the types of data used in the operation.

For example, consider the operation of addition. For two numbers, the operation will generate a sum. If the operands are strings, then the operation would produce a third string by concatenation. The process of making an operator to exhibit different behaviors in different instances is known as operator overloading.

Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface. This means that a general class of operations may be accessed in the same manner even though specific action associated
with each operation may differ. Polymorphism is extensively used in implementing inheritance.

Types of object-oriented languages

Significant object-oriented languages include:

Java, C++, C#, Python, R, PHP, Visual Basic.NET, JavaScript, Ruby, Perl, SIMSCRIPT, Object Pascal, Objective-C, Dart, Swift, Scala, Kotlin, Common Lisp, MATLAB, and Smalltalk.

OBJECT-ORIENTED DATABASE MANAGEMENT SYSTEM (OODBMS)

An object-oriented database management system represents information in the form of objects as used in object-oriented programming. OODBMS allows object-oriented programmers to develop products, store them as objects and replicate or modify existing objects to produce new ones within OODBMS. OODBMS allows programmers to enjoy the consistency that comes with one programming environment because the database is integrated with the programming language and uses the same representation model. Certain object-oriented databases are designed to work with object-oriented programming languages such as Delphi, Python, Java, Perl, Objective C and Visual Basic .NET.

Complex Objects and Composition

An object is a basic unit of Object-Oriented Programming and represents the real-life entities. Complex objects are the objects that are built from smaller or a collection of objects. For example, a mobile phone is made up of various objects like a camera, battery, screen, sensors, etc. In this article, we will understand the use and implementation of a complex object.

In object-oriented programming languages, object composition is used for objects that have a “has-a” relationship with each other. For example, a mobile has-a battery, has-a sensor, has-a screen, etc. Therefore, the complex object is called the whole or a parent object whereas a simpler object is often referred to as a child object. In this case, all the objects or components are the child objects which together make up the complex object(mobile).

Classes that have data members of built-in type work really well for simple classes. However, in real-world programming, the product or software comprises of many different smaller objects and classes. In the above example, the mobile phone consisted of various different objects that on a whole made up a complete mobile phone. Since each of those objects performs a different task, they all are maintained in different classes. Therefore, this concept of complex objects is being used in most of the real-world scenarios. The advantages of using this concept are:

- Each individual class can be simple and straightforward.
- One class can focus on performing one specific task and obtain one behavior.
- The class is easier to write, debug, understand, and usable by other programmers.
- While simpler classes can perform all the operations, the complex class can be designed to coordinate the data flow between the simpler classes.
- It lowers the overall complexity of the complex object because the main, task of the complex object would then be to delegate tasks to the sub-objects, who already know how to do them.

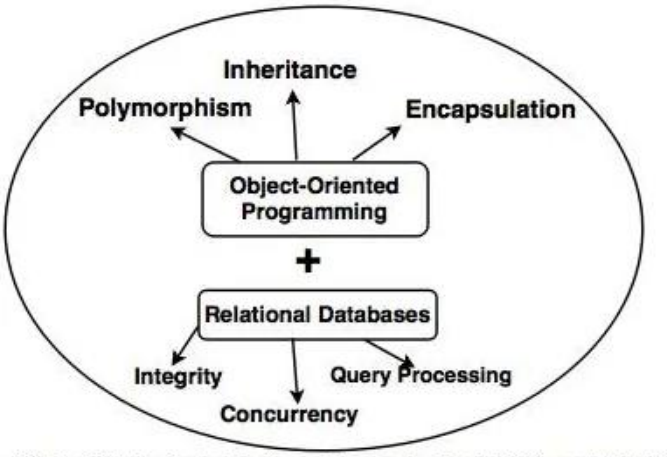
The most important advantage is if any changes have to be made in a child class, only the child class can be changed rather than changing the entire parent class.

- For example, if we want to change the battery class in the mobile object, with the help of composition, the changes are only made in the battery class and the entire mobile object works fine with the updated changes.

Concept of OODBMS

- Object oriented database systems are alternative to relational database and other database systems.
- In object oriented database, information is represented in the form of objects.
- Object oriented databases are exactly same as object oriented programming languages. If we can combine the features of relational model (transaction, concurrency, recovery) to object oriented databases, the resultant model is called as object oriented database model.

The composition of Object-Oriented Database can be shown in the following diagram:



Object-Oriented database

Some of the features of OODBMS

The features of OODMBS are Complexity, inheritance, encapsulation and persistency.

1. Complexity

OODBMS has the ability to represent the complex internal structure (of object) with multilevel complexity.

2. Inheritance

Creating a new object from an existing object in such a way that new object inherits all characteristics of an existing object.

3. Encapsulation

It is an data hiding concept in OOPL which binds the data and functions together which can manipulate data and not visible to outside world.

4. Persistency

OODBMS allows to create persistent object (Object remains in memory even after execution). This feature can automatically solve the problem of recovery and concurrency.

MODULE II

DESIGNING FORMS, REPORTS AND TRIGGERS IN OBJECT ORIENTED DATABASES

Forms and reports are an important part of the database application. Decision makers and clerical workers use forms and reports often. Designers use them to create an integrated application. Users can perform their tasks easily by using forms and reports.

Forms are used to collect data, display results of the queries, perform computations etc.

Reports are used to give the summary data. The report may be given as in graphical manner. Generally, it would occupy one page. Applications are built as a collection of related Forms and reports. All the users access the database using forms and reports. So, it should be designed to match the user's tasks.

Practices for effective design of forms and reports

Forms and Reports are the primary contact with the user. Before designing the form consider the following practices:

1. Analyze the user requirement first.
2. All the forms and reports within the application must be a consistent one.
3. The commands, Keystrokes, Icons should be used for the same purposes throughout the application.
4. Color and structure must be coordinated so users can understand the data easily on any form or report.

Human factors in design

In the current operating system the primary fact is that the users and not the application should always have control. For example, do not expect the users to enter the data. Instead make the users to select the data from the list. So based upon the user's choice various events are triggered.

Application must respond to the user's triggers or events by performing calculations, storing data and offer new choices. Both the design and color must be consistent throughout the application. If Application has multiple forms and reports organize them according to user tasks.

Window Controls

The components of the window consist of the frame, the title bar, a control menu box and various standard buttons. The frame can be resized by the user. The title for each form must be short and meaningful. The control menu box provides standard commands to move, resize and close the form. Common window buttons include Maximize and Minimize and Close. Scroll bars enable the users to scroll the form horizontally and vertically.

Menus are an important feature of any application. The most common menu is a drop-down menu displayed at the top of the screen. The menu contains list of actions. Mnemonic letters are generally underlined. (Example the F in the File command). Many applications also define shortcut keys. These commands generally include with Ctrl key combination. (Example Ctrl+C command to copy a selection.) Developers use a pop-up or shortcut menu. To trigger a pop-up menu click the right mouse button. The pop-up menu is usually context sensitive. A context-sensitive menu is one that changes depending on the object selected by the user.

Message box is an example for the pop-up menu. Message box is used to display short one-line messages (usually yes/no) responses from the user. In general, it is best to avoid the use of message boxes because they remove the control from the user.

User interface-web note

In many ways the web environment is similar to the standard Windows environment. All the web pages and forms are standardized because they are displayed the same way on almost any type of computer.

Clicking a hyperlink brings a new page, buttons are used to submit a form, and drop-down lists are used to select an item and so on.

User interface-accessibility issues

The greatest strengths of the windows interface is its graphical orientation. We can Form or make our applications accessible to many users.

Form layout

There are four basic types of forms. They are;

1. Tabular Forms
2. Single-row Forms
3. Sub form Forms.
4. Switchboard forms.

Tabular forms display data in rows and columns.

Single-row forms show data for one row at a time in which the designer can arrange the values in any format on the screen.

Sub form forms display data from two tables that have a one-to-many relationship.

Switchboard forms or menus direct the user to other forms and reports in the application.

Forms have several things in common. They have properties which are used to control the look and style of the form. Forms also have controls which includes labels and textboxes. Several events can occur in the forms. Opening and Closing are the basic events in the form. Although a form can have multiple controls only one control will have the focus. The control which has the focus will receive the keystrokes entered by the user. This control is highlighted with an outline or different color. The users can make use of the tab order. If we set the tab order means the users can easily move from one control to the next by just pressing the tabs.

TRIGGERS

What Is a Trigger?

This is a single event on a particular database element, like a table, can cause a chain reaction of many other events in the database. Let's look at some definitions and examples.

Definition and Examples

A trigger, in database terms, is a set of instructions that's activated (or we say it is 'fired') by some specific event, normally a command issued through the database's Data Manipulation Language (DML). DML is what we use to put data in, take data out, retrieve data from, and change data within a database.

Most DML instances contain at least four basic commands: SELECT, INSERT, UPDATE, and DELETE. Triggers can be set to fire BEFORE, AFTER, or INSTEAD OF any of these commands.

Triggers are most commonly used to automate related, repetitious tasks within a database and to ensure that data is consistent across the database wherever it's stored. Consider the following scenario:

John is a new employee at Super Cyber Store (SCS). Like most companies, SCS has a few different applications that will need access to John's information, like Human Resources, Payroll, and Benefits. Without a database trigger, the data entry person at SCS would have to enter all of John's personal data three times, once for each system. With a trigger, the database that holds the records for the three systems can be made to transfer all of John's information after it's entered one time. Once the data

entry person at SCS enters the data into the HR table in the database, a trigger inserts the same data into the payroll and benefits tables.

UML DIAGRAMS

UML is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.

- UML stands for **Unified Modeling Language**.
- UML is different from the other common programming languages such as C++, Java, COBOL, etc.
- UML is a pictorial language used to make software blueprints.
- UML can be described as a general purpose visual modeling language to visualize, specify, construct, and document software system.
- Although UML is generally used to model software systems, it is not limited within this boundary. It is also used to model non-software systems as well. For example, the process flow in a manufacturing unit, etc.

UML Standard Diagrams

We prepare UML diagrams to understand a complex system in a better and simple way. A single diagram is not enough to cover all the aspects of the system. UML defines various kinds of diagrams to cover most of the aspects of a system.

You can also create your own set of diagrams to meet your requirements. Diagrams are generally made in an incremental and iterative way.

There are two broad categories of diagrams and they are again divided into subcategories −

1. Structural Diagrams
2. Behavioral Diagrams

i). Structural Diagrams

The structural diagrams represent the static aspect of the system. These static aspects represent those parts of a diagram, which forms the main structure and are therefore stable.

These static parts are represented by classes, interfaces, objects, components, and nodes. The four structural diagrams are –

1. Class diagram
2. Object diagram
3. Component diagram
4. Deployment diagram

Class Diagram

Class diagrams are the most common diagrams used in UML. Class diagram consists of classes, interfaces, associations, and collaboration. Class diagrams basically represent the object-oriented view of a system, which is static in nature.

Object Diagram

Object diagrams can be described as an instance of class diagram. Thus, these diagrams are closer to real-life scenarios where we implement a system. Object diagrams are a set of objects and their relationship is just like class diagrams. They also represent the static view of the system.

Component Diagram

Component diagrams represent a set of components and their relationships. These components consist of classes, interfaces, or collaborations. Component diagrams represent the implementation view of a system. During the design phase, software artifacts (classes, interfaces, etc.) of a system are arranged in different groups depending upon their relationship. Now, these groups are known as components.

Deployment Diagram

Deployment diagrams are a set of nodes and their relationships. These nodes are physical entities where the components are deployed. Deployment diagrams are used for visualizing the deployment view of a system. This is generally used by the deployment team.

ii) Behavioral Diagrams

Any system can have two aspects, static and dynamic (changing/moving parts of a system). So, a model is considered as complete when both the aspects are fully covered.

UML has the following five types of behavioral diagrams −

1. Use case diagram
2. Sequence diagram
3. Collaboration diagram
4. Statechart diagram
5. Activity diagram

Use Case Diagram

Use case diagrams are a set of use cases, actors, and their relationships. They represent the use case view of a system.

A use case represents a particular functionality of a system. Hence, use case diagram is used to describe the relationships among the functionalities and their internal/external controllers. These controllers are known as **actors**.

Sequence Diagram

A sequence diagram is an interaction diagram. From the name, it is clear that the diagram deals with some sequences, which are the sequence of messages flowing from one object to another.

Interaction among the components of a system is very important from implementation and execution perspective. Sequence diagram is used to visualize the sequence of calls in a system to perform a specific functionality.

Collaboration Diagram

Collaboration diagram is another form of interaction diagram. It represents the structural organization of a system and the messages sent/received. Structural organization consists of objects and links.

The purpose of collaboration diagram is similar to sequence diagram. However, the specific purpose of collaboration diagram is to visualize the organization of objects and their interaction.

Statechart Diagram

Any real-time system is expected to be reacted by some kind of internal/external events. These events are responsible for state change of the system. State chart diagram is used to visualize the reaction of a system by internal/external factors.

Activity Diagram

Activity diagram describes the flow of control in a system. It consists of activities and links. The flow can be sequential, concurrent, or branched. Activities are nothing but the functions of a system. Numbers of activity diagrams are prepared to capture the entire flow in a system.

Activity diagrams are used to visualize the flow of controls in a system. This is prepared to have an idea of how the system will work when executed.

UML Structural diagram example:

Example of a class diagram

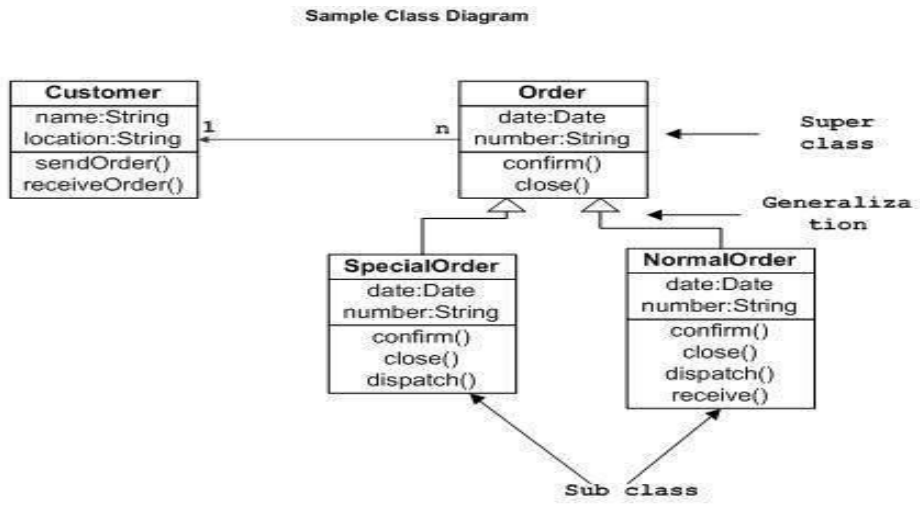
Consider this *Scenario*: a customer placed some Order of Textbooks from a publishing company, he placed these Order as Special Order and Normal Order.

As a system designer, draw a Class diagram to describe this Order Management System considering the following information:

Object	**Attribute**	**Function**
Customer	Name: String, Address: String	SendOrder(), ReceiveOrder()
Order	Date: Date, Number: String	Confirm(), Close()
SpecialOrder	Date: Date, Number: String	Confirm(), Dispatch(), Close()
NormalOder	Date: Date, Number: String	Confirm(), Dispatch(), Receive(), Close()

Answer:

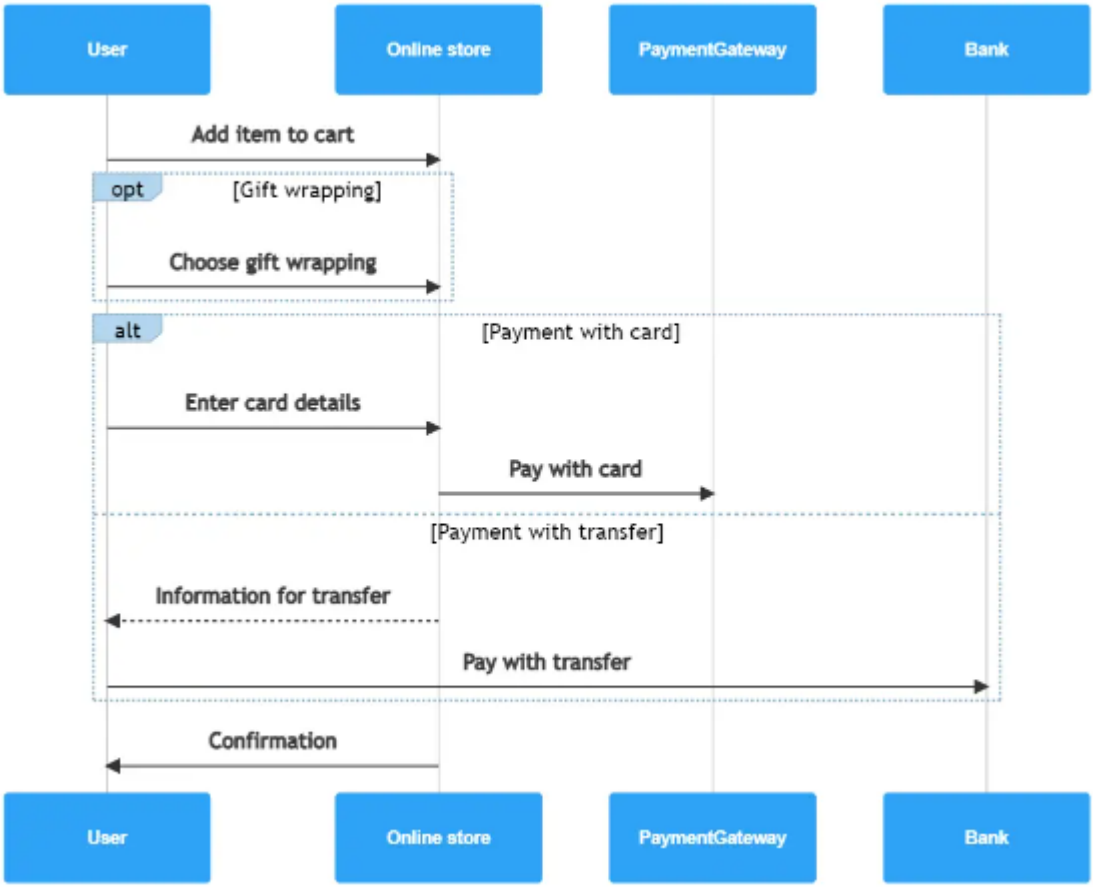
The following class diagram has been drawn considering all the points mentioned above.



UML Behavioral diagram example:

Example of sequence diagram:

1. Consider a customer buying things from online store:



2. UML activity diagram

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system.

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system.

The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc

Purpose of Activity Diagrams

The basic purposes of activity diagrams are similar to other four diagrams. It captures the dynamic behavior of the system. Other four diagrams are used to show the message flow from one object to another but activity diagram is used to show message flow from one activity to another.

The purpose of Activity diagram are as follows:

1. Draw the activity flow of a system.
2. Describe the sequence from one activity to another.
3. Describe the parallel, branched and concurrent flow of the system.

How to Draw an Activity Diagram?

Activity diagrams are mainly used as a flowchart that consists of activities performed by the system. Activity diagrams are not exactly flowcharts as they have some additional capabilities. These additional capabilities include branching, parallel flow, etc.

Before drawing an activity diagram, we should identify the following elements −

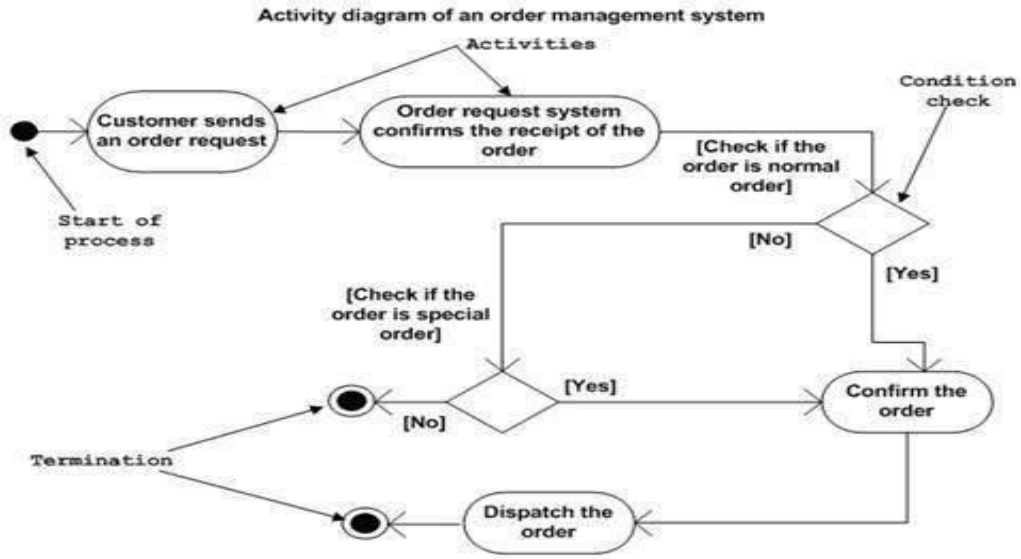
- Activities
- Association

- Conditions
- Constraints

Below is the sample of activity diagram considering an order management system as example:
The activity diagram is made to understand the flow of activities and is mainly used by the business users. Following diagram is drawn with the four main activities:

1) Send order by the customer
2) Receipt of the order
3) Confirm the order
4) Dispatch the order

After receiving the order request, condition checks are performed to check if it is normal or special order. After the type of order is identified, dispatch activity is performed and that is marked as the termination of the process.



ENTITY RELATIONSHIP MODEL

What is an ERD?

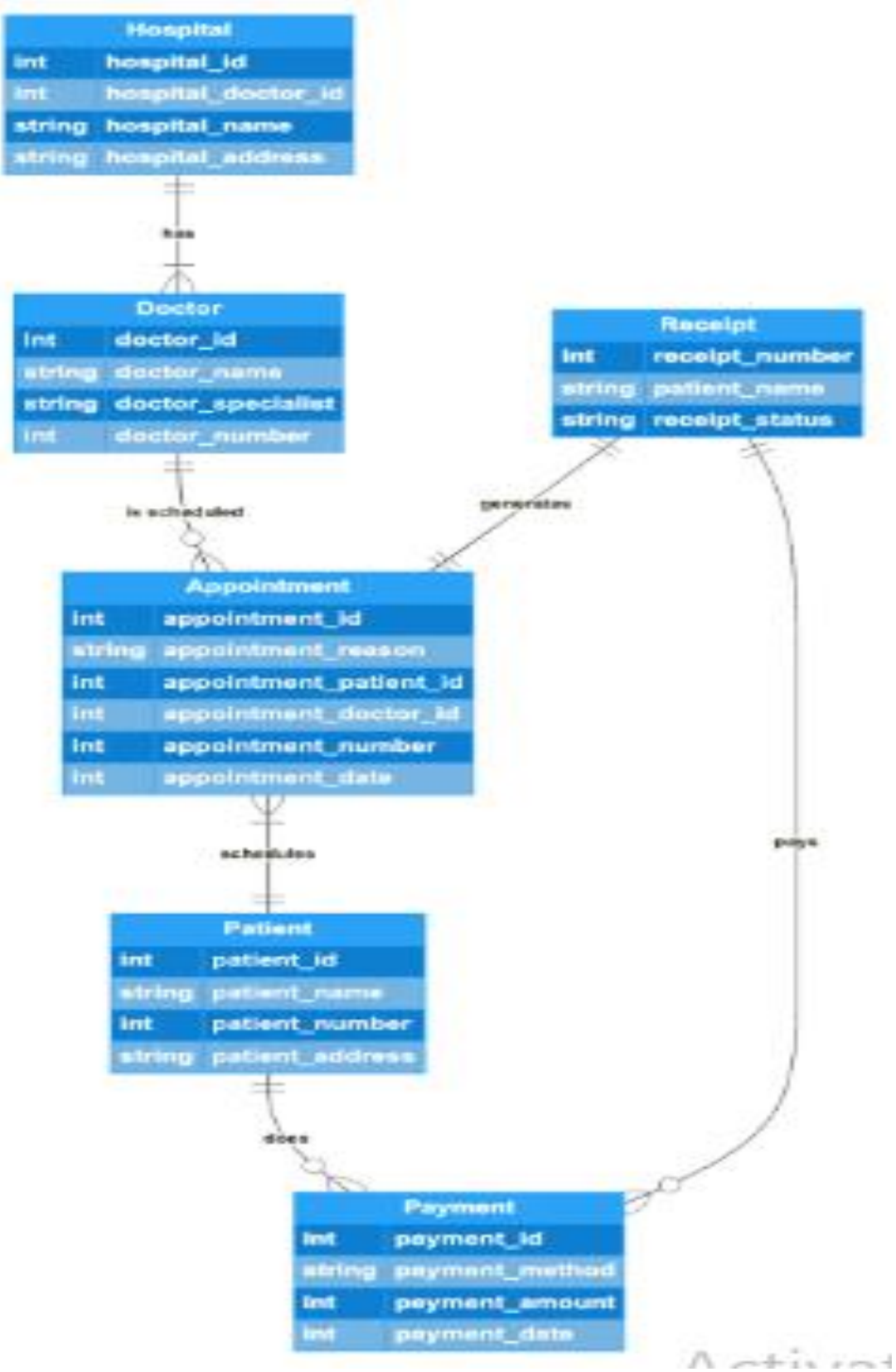
ERD is short for Entity Relationship Diagram, and they are used to plan and model databases before their creation. ER diagrams are an offshoot of the entity-relationship model, which shows how entities interact in a database:

ERD notations

There are 6 notation types used to make ERDs that serve different functions for the reader. Here are a few of the most common ones.

Chen notation – This notation type is more detailed than others. Users can identify entities and relationships as weak or strong, and attributes can be classified in additional ways.

Crow's foot notation – Crow's notation is one of the more detailed notation styles. One difference is the presence of multiplicities, which tell the reader how many instances interact with other instances.


Hospital
int hospital_id
int hospital_doctor_id
string hospital_name
string hospital_address
has
Doctor
Int doctor_id
string doctor_name
string doctor_specialist
int doctor_number
is scheduled
Receipt
int receipt_number
string patient_name
string receipt_status
generates
Appointment
int appointment_id
string appointment_reason
int appointment_patient_id
int appointment_doctor_id
int appointment_number
int appointment_date
schedules
pays
Patient
int patient_id
string patient_name
int patient_number
string patient_address
does
Payment
int payment_id
string payment_method
int peyment_amount
int payment_date

Barker notation – The most recognized form of ERD notation, this style shows relationship degrees, optionality, and attributes as mandatory or optional.

Common ERD components

Entities – Real world objects, such as users, are represented with rectangles. There are several kinds of entity notations.

- **Strong entity** – Shown in one rectangle. Strong entities stand alone, and often have weaker entities relying on them.
- **Weak entity** – Two rectangles, one slightly smaller than the other. Weak entities rely on another entity.
- **Associative entity** – A diamond inside a rectangle. Associative entities show the relation of several entity types, as well as attributes specific to each interaction.

Attributes – These are the qualities or characteristics entities have, and are shown in ovals.

- **Attribute** – In a single oval, attributes are qualities entities have in one to one or many to many relationships.
- **Multivalued attribute** – Two ovals, one inside the other. A multivalued attribute is able to take on more than one value.
- **Derived attribute** – Represented by an oval with a dashed or dotted line. These attributes are values that are calculated from other, associated attributes.

Keys – Keys are a way to categorize attributes using lines and symbols, and they connect relevant fields. Keys clearly and concisely show information, which keeps the diagram from becoming too cluttered. There are two types of keys.

- **Primary keys** – One or more attributes that define exactly one instance of an entity
- **Foreign keys** – Created in a one to one or a one to many relationship, when one entity relates to another entity.

Relationships/Cardinality – Represent ed with diamonds, relationships are how entities interact with each other. There are 3 common relationship types.

- **1:1** – Entities share exactly 1 instance with each other.
- **1:M** – One relationship of an entity has multiple relationships with another.
- **M:N** – Two entities both share many instances with another.

What is the difference between UML and ERD?

The main difference between UML and ER diagrams is that UML is a language used to create diagrams, whereas ERDs are a type of diagram. UML is used for planning software development, and is used in many different diagrams for various purposes. ER diagrams do not focus on the software, but rather the modelling of databases, which are usually part of a software system. UML diagrams are broader, and have many uses, and ERDs only have one purpose.

Here are some other notable differences between ER diagrams and UML.

	UML	ERD
Full form	Unified Modeling Language	Entity Relationship Diagram
Definition	An easy to read system of symbols, shapes, and notations used in software system modeling and planning	A diagram that shows real-world items that exist in a database, and uses symbols and shapes to show relationships, attributes, and other important details
Key attributes	Class diagrams Object diagrams Sequence diagrams Activity diagrams Communication diagrams	Entities Attributes Cardinality Ordinality Number of relationship instances
Uses	Plan and model software systems Show how entities operate within a system, with all possible interactions	Plan databases Ensure all entities function properly Defines attributes of entities

MOBULE III: GROUP

UNDERSTANDING FILE STRUCTURE AND PHYSICAL STORAGE

Storage System in DBMS

A database system provides an ultimate view of the stored data. However, data in the form of bits, bytes get stored in different storage devices. In this section, we will take an overview of various types of storage devices that are used for accessing and storing data.

Types of Data Storage

For storing the data, there are different types of storage options available. These storage types differ from one another as per the speed and accessibility. There are the following types of storage devices used for storing the data:

1. Primary storage
2. Secondary storage
3. Tertiary storage
4. Offline storage

1. Primary Storage

Primary storage is also called main memory. Main memory is directly or indirectly connected to the central processing unit via the memory bus. The CPU constantly reads instructions stored on it and executes them as needed. In primary storage, we can read and write data quickly. We can identify different types of primary storage.

- RAM — RAM is called Random access memory. Because any of the data in random access memory can be accessed just as fast as any of other data. RAM stores data till the computer is working. When the computer is turn switched off, data is erased. RAM is volatile because when there is a power failure data stored data is lost.

- ROM — ROM means Read-Only Memory. ROM is used as the computer begins to boot up. Small programs like firmware are often stored in ROM chips on hardware devices. ROM memory cannot be easily or quickly overwritten or modified. So ROM is non — volatile. ROM is hold data when power is turned off.

- Cache — Cache memory is small in size and volatile memory. It provides high-speed data access to a processor and Cache stores frequently used computer programs, applications and data. Cache makes data retrieval easy and efficient.

2. Secondary Storage

Secondary storage is non-volatile, low cost, high capacity, portable or not. This is long-term storage. If there is secondary storage in the computer we can store data permanently. Secondary storage is also known as additional memory or auxiliary memory. Secondary storage could be internal or external. Hard disk drive, the tape disk drive, CD- ROM, BLU — ray, DVD, and floppy disk drive are used as internal storage. USB flash drives, and plug and play devices are used as external storage.

Example for secondary storage devices

- Hard disk drives
- Cloud storage
- CD — ROM
- DVD drives
- Blu-ray drives
- USB Flash drives
- SD cards
- Tape drives
- Zip and Jaz drives

The hard disk drive is the primary and usually largest data storage device on your computer. It can store data from 160 gigabytes to 2 terabytes. A hard disk speed is a speed at which content can be read and written on a hard drive. The set rotation speed of the hard drive is from 4500 to 7200rpm. Disk access time is measured in milliseconds. There are two types of hard disks:

- Internal hard disk
- External hard disk

Internal hard disk is not portable, less expensive, fast, and big but external hard disk is portable, more expensive, slow, and small in size.

Flash Memory: A flash memory stores data in USB (Universal Serial Bus) keys which are further plugged into the USB slots of a computer system. These USB keys help transfer data to a computer system, but it varies in size limits. Unlike the main memory, it is possible to get back the stored data which may be lost due to a power cut or other reasons. This type of memory storage is most commonly used in the server systems for caching the frequently used data. This leads the systems towards high performance and is capable of storing large amounts of databases than the main memory.

Magnetic Disk Storage: This type of storage media is also known as online storage media. A magnetic disk is used for storing the data for a long time. It is capable of storing an entire database. It is the responsibility of the computer system to make availability of the data from a disk to the main memory for further accessing. Also, if the system performs any operation over the data, the modified data should be written back to the disk. The tremendous capability of a magnetic disk is that it does not affect the data due to a system crash or failure, but a disk failure can easily ruin as well as destroy the stored data.

3. Tertiary Storage

The removable storage devices are used as tertiary storage devices. Tertiary storage provides the third tier of storage. They are primarily useful for large data stores, accessed without human intervention. Tertiary storage includes a robotic mechanism that will mount and dismount removable mass storage media into a storage device as required by the system. When a computer needs to read information from the tertiary storage, it first consults a catalog database to determine which tape or disc contains the information. The computer will then use a robotic arm to fetch the medium and place it in a drive. When the computer finishes reading the instruct information. The robotic arm will return the medium to its place in the library. Tertiary storages are inexpensive and removable. Examples of tertiary storage devices are:

i. **Optical Storage:** An optical disc is any storage media that is digitally contained in digital format and readable by laser assembly which is considered an optical media. The most common types of optical media are, Blu-ray disc, Compact Disc (CD), Digital Versatile Disc (DVD).
   An optical storage can store megabytes or gigabytes of data. A Compact Disk or CD can store 700 megabytes of data with a playtime of around 80 minutes. On the other hand, a Digital Video Disk or a DVD can store 4.7 or 8.5 gigabytes of data on each side of the disk.
ii. **Tape Storage:** A magnetically coated strip of plastic on which data can be encoded. Tapes for computers are similar to tapes used to store music. Tape is inexpensive than other storage mediums. That is commonly used for backup.

4. Offline storage

Off-line storage is also known as disconnected storage. Off-line is a computer data storage on a medium or a device that is not under the control of a processing unit. Before re-accessing a computer,

it must be entered or connected by a human operator. The stored data are remaining permanently in an offline storage device even if the computer is disconnected from it. Offline storage devices are portable and can be used in different computer systems. This is often used for transferring data.

Examples are:

- **Zip diskette**

Zip diskette is a hardware data storage device. It performs like a standard 1.44'' floppy drive. They can hold up to 100MB of data or 250MB of data on mew drives. Now these are less popular because of their small storage capacity.

- **USB Flash drive**

They are small and portable memory cards that act like portable hard drives.it can be connected to the computer by the computer's USB port. There are different sizes of flash drives such as 2GB, 4GB, and up to 256GB.

- **Memory card**

A memory card is an electronic flash memory storage disk. It is commonly used in consumer electronic devices such as digital camera's MP3 players, mobile phones, and other small portable devices. Memory cards are usually read by connecting the devices containing the card to your computer or by using a USB card reader.

Characteristics of computer storage devices

1. Volatility — among computer storage devices some are volatile and others are non — volatile. In volatile storage devices, when the computer is turned off, stored data will be lost. As an example-RAM. In non-volatile storage devices even if power is turned off, data will store. Most secondary, tertiary, and off-line storage is non-volatile.
2. Speed — speed measures that how much time takes a device to read or write data. Most internal storage devices are speed than external storage devices as an example internal hard drive is speed than an external hard drive.
3. Portability — most storage devices can carry anywhere for example -USB, Optical disks. These are small in size as well removable but some storage devices are located in the computer or any other devices. These are normally big and not removable. Internal storage devices are difficult to carry anywhere.
4. Cost — capacity, portability, the durability of storage devices are affected by cost. Storage devices which can store a large capacity of data are expensive.
5. Durability — if something is durable. It can be able to withstand physical movements. Especially portable storage devices should be durable. These are carried anywhere. So portable storage devices should be durable to prevent easy damage.

FILE ORGANIZATION IN DBMS

To access these files, we need to store them in certain order so that it will be easy to fetch the records. It is same as indexes in the books, or catalogues in the library, which helps us to find required topics or books respectively. Therefore, storing files in certain order is called file organization.

File organization refers to the way data is stored in a file. File organization is very important because it determines the methods of access, efficiency, flexibility and storage devices to use. There are various methods of file organizations. Hence it is up to the programmer to decide the best suited file organization method depending on his requirement.

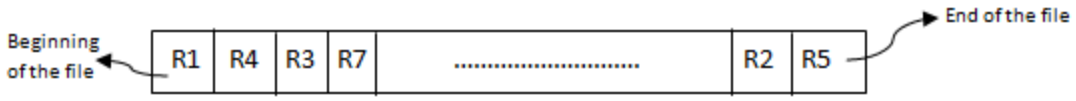
Some of the methods of file organizations are:

1. Sequential File Organization
2. Indexed Sequential Access Method
3. Heap File Organization
4. Hash/Direct File Organization
5. B+ Tree File Organization
6. Cluster File Organization

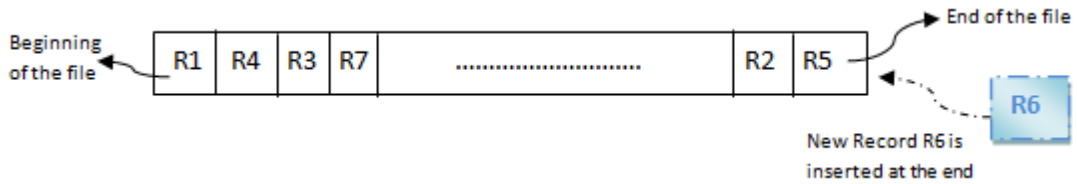
1. Sequential File Organization

It is one of the simple methods of file organization. Here each file/records are stored one after the other in a sequential manner. This can be achieved in two ways:

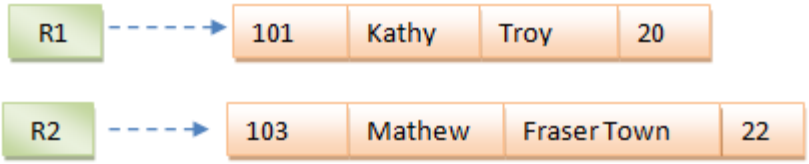
- Records are stored one after the other as they are inserted into the tables. This method is called **pile file method**. When a new record is inserted, it is placed at the end of the file. In the case of any modification or deletion of record, the record will be searched in the memory blocks. Once it is found, it will be marked for deleting and new block of record is entered.



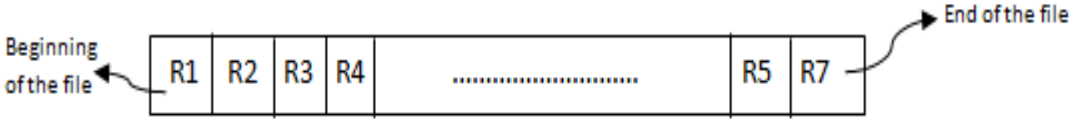
Inserting a new record:



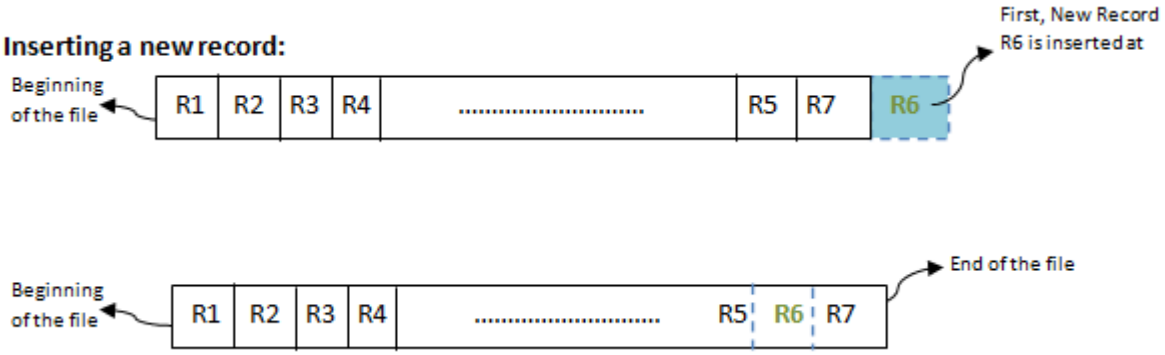
In the diagram above, R1, R2, R3 etc are the records. They contain all the attribute of a row. i.e.; when we say student record, it will have his id, name, address, course, DOB etc. Similarly R1, R2, R3 etc can be considered as one full set of attributes.



- In the second method, records are sorted (either ascending or descending) each time they are inserted into the system. This method is called **sorted file method**. Sorting of records may be based on the primary key or on any other columns. Whenever a new record is inserted, it will be inserted at the end of the file and then it will sort – ascending or descending based on key value and placed at the correct position. In the case of update, it will update the record and then sort the file to place the updated record in the right place. Same is the case with delete.



Inserting a new record:



Advantages of Sequential File Organization

- The design is very simple compared other file organization. There is no much effort involved to store the data.
- When there are large volumes of data, this method is very fast and efficient. This method is helpful when most of the records have to be accessed like calculating the grade of a student, generating the salary slips etc where we use all the records for our calculations
- This method is good in case of report generation or statistical calculations.
- These files can be stored in magnetic tapes which are comparatively cheap.

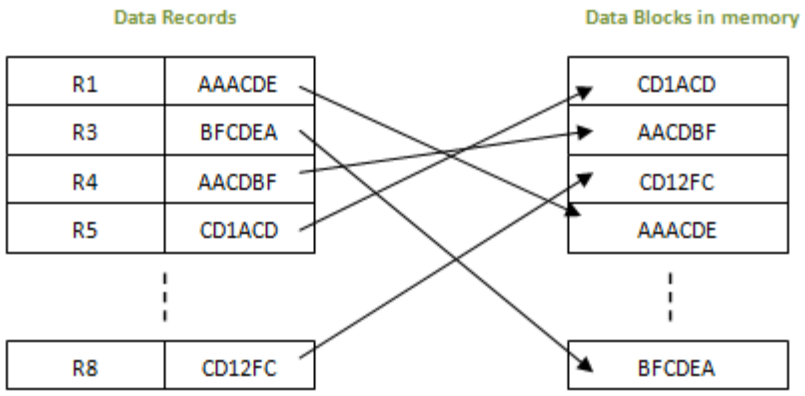
Disadvantages of Sequential File Organization

- Sorted file method always involves the effort for sorting the record. Each time any insert/update/ delete transaction is performed, file is sorted. Hence identifying the

record, inserting/ updating/ deleting the record, and then sorting them always takes some time and may make system slow.

2. Indexed Sequential Access Method (ISAM)

This is an advanced sequential file organization method. Here records are stored in order of primary key in the file. Using the primary key, the records are sorted. For each primary key, an index value is generated and mapped with the record. This index is nothing but the address of record in the file.



In this method, if any record has to be retrieved, based on its index value, the data block address is fetched and the record is retrieved from memory.

Advantages of ISAM

- Since each record has its data block address, searching for a record in larger database is easy and quick. There is no extra effort to search records. But proper primary key has to be selected to make ISAM efficient.
- This method gives flexibility of using any column as key field and index will be generated based on that. In addition to the primary key and its index, we can have index generated for other fields too. Hence searching becomes more efficient, if there is search based on columns other than primary key.
- It supports range retrieval, partial retrieval of records. Since the index is based on the key value, we can retrieve the data for the given range of values. In the same way, when a partial key value is provided, say student names starting with 'JA' can also be searched easily.

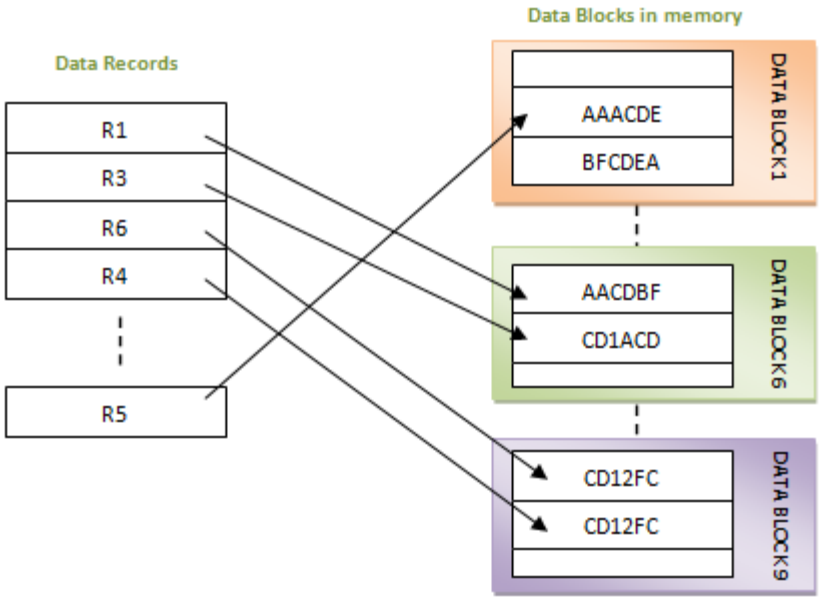
Disadvantages of ISAM

- An extra cost to maintain index has to be afforded. i.e.; we need to have extra space in the disk to store this index value. When there is multiple key-index combinations, the disk space will also increase.

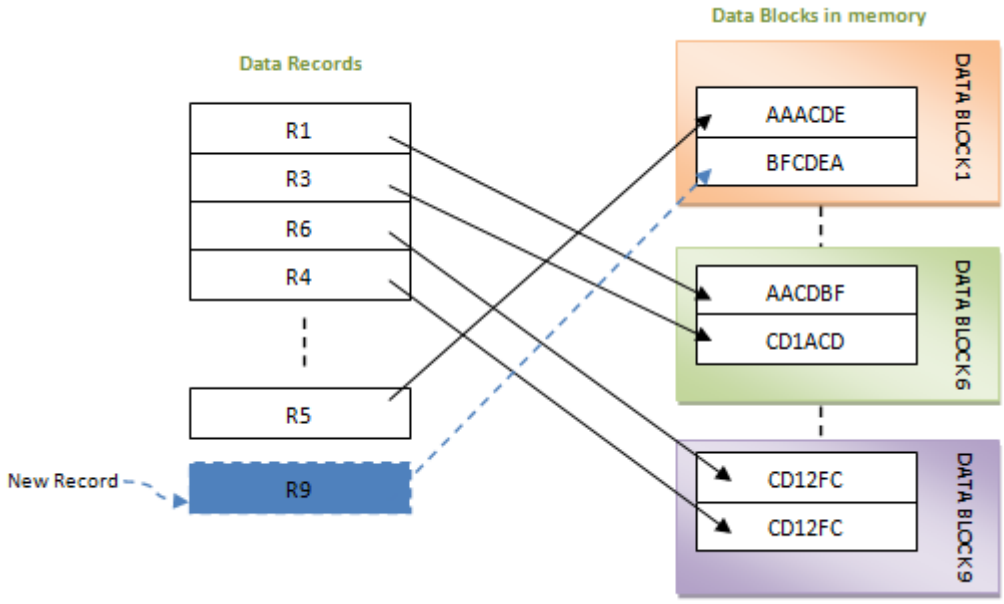
As the new records are inserted, these files have to be restructured to maintain the sequence. Similarly, when the record is deleted, the space used by it needs to be released. Else, the performance of the database will slow down.

3. Heap File Organization

This is the simplest form of file organization. Here records are inserted at the end of the file as and when they are inserted. There is no sorting or ordering of the records. Once the data block is full, the next record is stored in the new block. This new block need not be the very next block. This method can select any block in the memory to store the new records. It is similar to pile file in the sequential method, but here data blocks are not selected sequentially. They can be any data blocks in the memory. It is the responsibility of the DBMS to store the records and manage them.



If a new record is inserted, then in the above case it will be inserted into data block 1.



When a record has to be retrieved from the database, in this method, we need to traverse from the beginning of the file till we get the requested record. Hence fetching the records in very huge tables, it is time consuming. This is because there is no sorting or ordering of the records. We need to check all the data.

Similarly if we want to delete or update a record, first we need to search for the record. Again, searching a record is similar to retrieving it- start from the beginning of the file till the record is fetched. If it is a small file, it can be fetched quickly. But larger the file, greater amount of time needs to be spent in fetching.

In addition, while deleting a record, the record will be deleted from the data block. But it will not be freed and it cannot be re-used. Hence as the number of record increases, the memory size also increases and hence the efficiency. For the database to perform better, DBA has to free this unused memory periodically.

Advantages of Heap File Organization

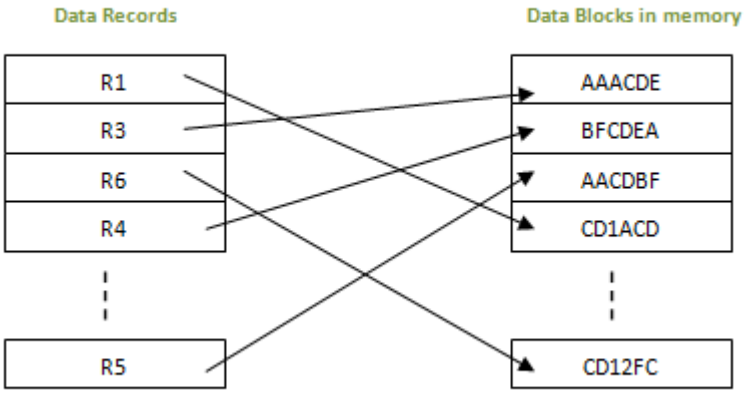
- Very good method of file organization for bulk insertion. i.e.; when there is a huge number of data needs to load into the database at a time, then this method of file organization is best suited. They are simply inserted one after the other in the memory blocks.
- It is suited for very small files as the fetching of records is faster in them. As the file size grows, linear search for the record becomes time consuming.

Disadvantages of Heap File Organization

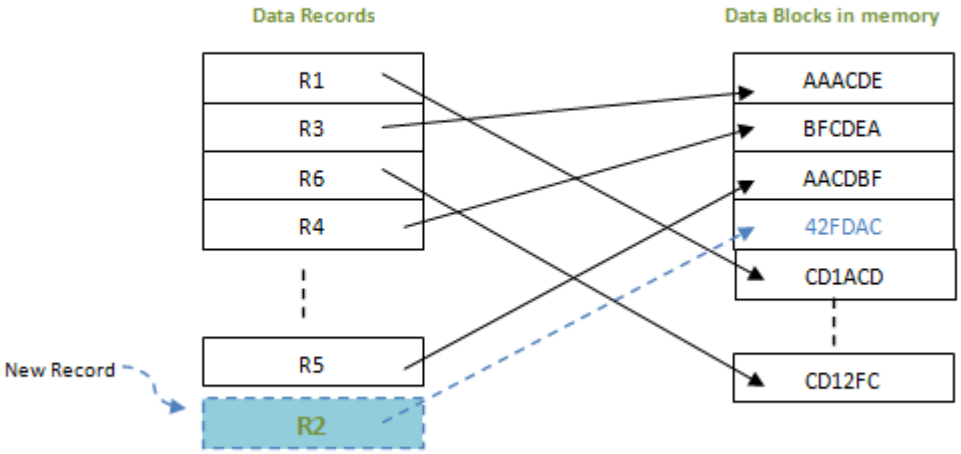
- This method is inefficient for larger databases as it takes time to search/modify the record.
- Proper memory management is required to boost the performance. Otherwise there would be lots of unused memory blocks lying and memory size will simply be growing.

4. Hash/Direct File Organization

In this method of file organization, hash function is used to calculate the address of the block to store the records. The hash function can be any simple or complex mathematical function. The hash function is applied on some columns/attributes – either key or non-key columns to get the block address. Hence each record is stored randomly irrespective of the order they come. Hence this method is also known as Direct or Random file organization. If the hash function is generated on key column, then that column is called hash key, and if hash function is generated on non-key column, then the column is hash column.



When a record has to be retrieved, based on the hash key column, the address is generated and directly from that address whole record is retrieved. Here no effort to traverse through whole file. Similarly when a new record has to be inserted, the address is generated by hash key and record is directly inserted. Same is the case with update and delete. There is no effort for searching the entire file nor sorting the files. Each record will be stored randomly in the memory.



These types of file organizations are useful in online transaction systems, where retrieval or insertion/updation should be faster.

Advantages of Hash File Organization

- Records need not be sorted after any of the transaction. Hence the effort of sorting is reduced in this method.
- Since block address is known by hash function, accessing any record is very faster. Similarly updating or deleting a record is also very quick.

- This method can handle multiple transactions as each record is independent of other. i.e.; since there is no dependency on storage location for each record, multiple records can be accessed at the same time.
- It is suitable for online transaction systems like online banking, ticket booking system etc.

Disadvantages of Hash File Organization

- This method may accidentally delete the data. For example, In Student table, when hash field is on the STD_NAME column and there are two same names – 'Antony', then same address is generated. In such case, older record will be overwritten by newer. So there will be data loss. Thus hash columns needs to be selected with utmost care. Also, correct backup and recovery mechanism has to be established.
- Since all the records are randomly stored, they are scattered in the memory. Hence memory is not efficiently used.
- If we are searching for range of data, then this method is not suitable. Because, each record will be stored at random address. Hence range search will not give the correct address range and searching will be inefficient. For example, searching the employees with salary from 20K to 30K will be efficient.
- Searching for records with exact name or value will be efficient. If the Student name starting with 'B' will not be efficient as it does not give the exact name of the student.
- If there is a search on some columns which is not a hash column, then the search will not be efficient. This method is efficient only when the search is done on hash column. Otherwise, it will not be able find the correct address of the data.
- If there is multiple hash columns – say name and phone number of a person, to generate the address, and if we are searching any record using phone or name alone will not give correct results.
- If these hash columns are frequently updated, then the data block address is also changed accordingly. Each update will generate new address. This is also not acceptable.

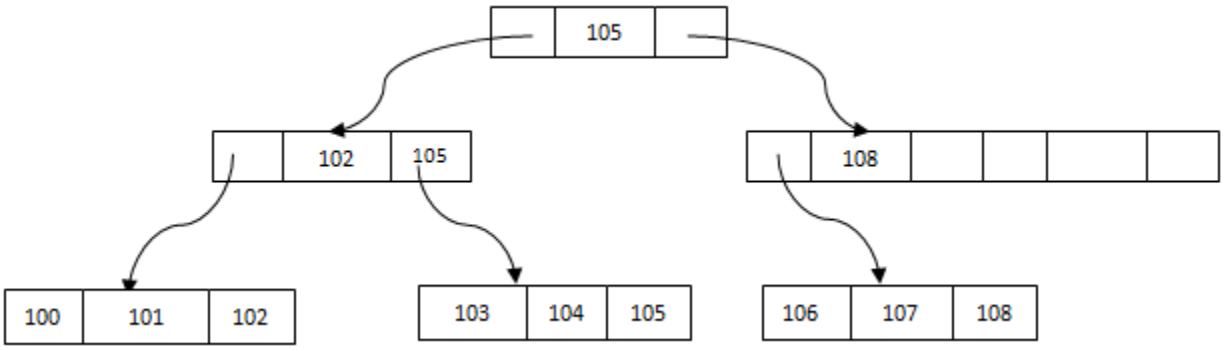
Hardware and software required for the memory management are costlier in this case. Complex programs needs to be written to make this method efficient.

5. B+ Tree File Organization

B+ Tree is an advanced method of ISAM file organization. It uses the same concept of key-index, but in a tree like structure. B+ tree is similar to binary search tree, but it can have more than two leaf nodes. It stores all the records only at the leaf node. Intermediary nodes will have pointers to the leaf nodes. They do not contain any data/records.

Consider a student table below. The key value here is STUDENT_ID. And each record contains the details of each student along with its key value and the index/pointer to the next value. In a B+ tree it can be represented as below.

STUDENT		
STUDENT_ID	STUDENT_NAME	ADDRESS
100	Joseph	Alaiedon Township
101	Allen	Fraser Township
102	Chris	Clinton Township
103	Patty	Troy
104	Jack	Fraser Township
105	Jessica	Clinton Township
106	James	Troy
107	Antony	Alaiedon Township
108	Jacob	Troy



Please note that the leaf node 100 means, it has name and address of student with ID 100, as we saw in R1, R2, R3 etc above.

From the above B+ tree structure, it is evident that

- There is one main node called root of the tree – 105 is the root here.
- There is an intermediary layer with nodes. They do not have actual records stored. They are all pointers to the leaf node. Only the leaf node contains the data in sorted order.
- The nodes to the left of the root nodes have prior values of root and nodes to the right have next values of the root. i.e.; 102 and 108 respectively.
- There is one final node, called leaf node, which has only values. i.e.; 100, 101, 103, 104, 106 and 107
- All the leaf nodes are balanced – all the leaf nodes at same distance from the root node. Hence searching any record is easier.
- Searching any record is linear in this case. Any record can be traversed through single path and accessed easily.

- Since the intermediary nodes have only pointers to the leaf node, the tree structure is of shorter height. Shorter the height, faster is the traversal and hence the retrieval of records.

Advantages of B+ Trees

- Since all records are stored only in the leaf node and are sorted sequential linked list, searching is becomes very easy.
- Using B+, we can retrieve range retrieval or partial retrieval. Traversing through the tree structure makes this easier and quicker.
- As the number of record increases/decreases, B+ tree structure grows/shrinks. There is no restriction on B+ tree size, like we have in ISAM.
- Since it is a balance tree structure, any insert/ delete/ update does not affect the performance.
- Since we have all the data stored in the leaf nodes and more branching of internal nodes makes height of the tree shorter. This reduces disk I/O. Hence it works well in secondary storage devices.

Disadvantages of B+ Trees

- This method is less efficient for static tables.

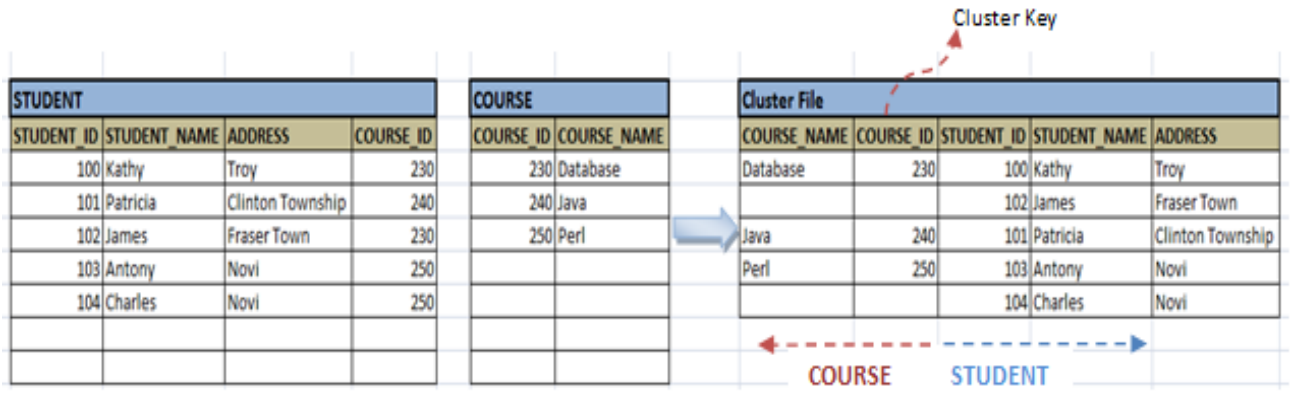
6. Cluster File Organization

In all the file organization methods described above, each file contains single table and are all stored in different ways in the memory. In real life situation, retrieving records from single table is comparatively less. Most of the cases, we need to combine/join two or more related tables and retrieve the data. In such cases, above all methods will not be faster to give the result. Those methods have to traverse each table at a time and then combine the results of each to give the requested result. This is obvious that the time taken for this is more. So what could be done to overcome this situation?

Another method of file organization – Cluster File Organization is introduced to handle above situation. In this method two or more table which are frequently used to join and get the results are stored in the same file called clusters. These files will have two or more tables in the same data block and the key columns which map these tables are stored only once. This method hence reduces the cost of searching for various records in different files. All the records are found at one place and hence making search efficient.

For example, we want to see the students who have taken particular course. The tables are shown in below diagram. We can see there are two students who have opted for ‘Database’ and

'Perl' course each. Though it is stored in separate tables in logical view, when it is stored in physical view, we have combined them. This can be seen in cluster file below. This is the result of join. So do not have to put any effort or time for joining. Hence it will give faster results.



If we have to insert or update or delete any record, we can directly do so. Here data are sorted based on the primary key or the key with which we are searching the data. Also, clusters are formed based on the join condition. The key with which we are joining the tables is known as cluster key.

Clustering of tables are done when

- There is a frequent need for joining the tables with same condition. Also, these joins will result in only few records from both tables. i.e.; in above example, we are retrieving the records for only particular course; not for the entire course. If all the records from any one of the table is used in the join condition, then this method is not efficient.
- If tables are joined once in a while or full table scan of any one the table in involved in the query, then we do not cluster the tables.
- If there is 1: M relationship between the tables, then we can cluster the tables. In above case for each course, we have many students opted for. Hence we have clustered.

There are two types of cluster file organization

- **Indexed Clusters: -** Here records are grouped based on the cluster key and stored together. Our example above to illustrate STUDENT-COURSE cluster is an indexed cluster. The records are grouped based on the cluster key – COURSE_ID and all the related records are stored together. This method is followed when there is retrieval of data for range of cluster key values or when there is a huge data growth in the clusters. That means, if we have to select the students who are attending the course with COURSE_ID 230-240 or there is a large number of students attending the same course, say 250.

- **Hash Clusters: -** This is also similar to indexed cluster. Here instead of storing the records based on the cluster key, we generate the hash key value for the cluster key and store the records with same hash key value together in the memory disk.

Advantages of Clustered File Organization

- This method is best suited when there is frequent request for joining the tables with same joining condition.
- When there is a 1:M mapping between the tables, it results efficiently

Disadvantages of Clustered File Organization

- This method is not suitable for very large databases since the performance of this method on them is low.
- We cannot use this clusters, if there is any change is joining condition. If the joining condition changes, the traversing the file takes lot of time.
- This method is not suitable for less frequently joined tables or tables with 1:1 conditions.

Data –dictionary storage concept

A data dictionary is like the A-Z dictionary of the relational database system holding all information of each relation in the database.

The types of information a system must store are:

1. Name of the relations
2. Name of the attributes of each relation
3. Lengths and domains of attributes
4. Name and definitions of the views defined on the database
5. Various integrity constraints
6. Objected Oriented database

MODULE IV

INDEXING AND HASHING

Many queries reference only a small proportion of the records in a file. For example, a query like "Find the balance of account number 101010" references only a fraction of the account records. It is inefficient for the system to read every record and to check the account number field for the value “101010”. Ideally the system should be able to locate these records directly. To allow these forms of access, additional structures (like index) need to be designed to associate with files.

Index.

An index is a record structure that provides easy access to stored records. An index for a file in a database system works in much the same way as the index in textbook. If we want to learn about a particular topic (specified by a word or a phrase) in a textbook, we can search for the topic in the index at the back of the book, find the pages where it occurs, and then read the pages to find the information we are looking for. Additionally, the words in the index are in sorted order, making it easy to find the

word we are looking for. Moreover, the index is much smaller than the book, further reducing the effort needed to find the words we are looking for.

Database-system indices play the same role as book indices in libraries. For example, to **retrieve** an account record given the **account number**, the database system would look up an index to find on which disk block the corresponding record resides, and then fetch the disk block, to get the account record.

Basic kinds of indices

There are two kinds of indices: (Q)

1. Ordered indices: these are indices that are based on a sorted ordering of the values.
2. Hash indices: these indices are based on a uniform distribution of values across a range of buckets bucket to which a value is assigned is determined by a function, called a hash function.

Each of the techniques must be evaluated on the basis of the following factors:

- Access types: The types of access that are supported efficiently. **Access types** can include **finding** records with a specified attribute value and finding records whose attribute values fall in a specified range.
- Access time: is the time it takes to find a particular data item, or set of items, using the technique in question.
- Insertion time: is the time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
- Deletion time: The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- Space overhead: The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

An **attribute** or set of attributes used to look up records in a file is called a **search key**. **Note** that this definition of key differs from that used in primary key, candidate key, and super-key.

Note: Hashing *is an advanced searching technique, i.e large data is made into small data items and stored in a table. But* ***indexing*** *and binary searching comes under searching in a linear manner.*

What is Transaction?

A transaction can be defined as a group of tasks. A single task is the minimum processing unit which cannot be divided further.

Let's take an example of a simple transaction. Suppose a bank employee transfers Rs 500 from A's account to B's account. This very simple and small transaction involves several low-level tasks.

A's Account

Open_Account(A)

Old_Balance = A.balance

New_Balance = Old_Balance - 500

A.balance = New_Balance

Close_Account(A)

B's Account

Open_Account(B)

Old_Balance = B.balance

New_Balance = Old_Balance + 500

B.balance = New_Balance

Close_Account(B)

ACID Properties

A transaction is a very small unit of a program and it may contain several lowlevel tasks. A transaction in a database system must maintain Atomicity, Consistency, Isolation, and Durability − commonly known as ACID properties − in order to ensure accuracy, completeness, and data integrity.

- Atomicity − This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none. There must be no state in a database where a transaction is left partially completed. States should be defined either before the execution of the transaction or after the execution/abortion/failure of the transaction.
- Consistency − The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well.
- Durability − The database should be durable enough to hold all its latest updates even if the system fails or restarts. If a transaction updates a chunk of data in a database and commits, then the database will hold the modified data. If a transaction commits but the system fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.
- Isolation − In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.

Concurrent executions, serialization recoverability and isolation

i. Concurrent Execution in DBMS

In a multi-user system, multiple users can access and use the same database at one time, which is known as the concurrent execution of the database. It means that the same database is executed simultaneously on a multi-user system by different users.

ii. Recoverable Schedules

If any transaction that performs a dirty read operation from an uncommitted transaction and also its committed operation becomes delayed till the uncommitted transaction is either committed or rollback such type of schedules is called as Recoverable Schedules.

Example

Let us consider two transaction schedules as given below −

T1	T2
Read(A)	
Write(A)	
-	Read(A) ///Dirty Read
-	Write(A)
-	
-	
Commit	
	Commit // delayed

The above schedule is a recoverable schedule because of the reasons mentioned below −

i. The transaction T2 performs dirty read operation on A.
ii. The commit operation of transaction T2 is delayed until transaction T1 commits or rollback.
iii. Transaction commits later.
iv. In the above schedule transaction T2 is now allowed to commit whereas T1 is not yet committed.
v. In this case transaction T1 is failed, and transaction T2 still has a chance to recover by rollback.

iii. Database isolation refers to the ability of a database to allow a transaction to execute as if there are no other concurrently running transactions (even though in reality there can be a large number of concurrently running transactions). The overarching goal is to prevent reads and writes of temporary, aborted, or otherwise incorrect data written by concurrent transactions.

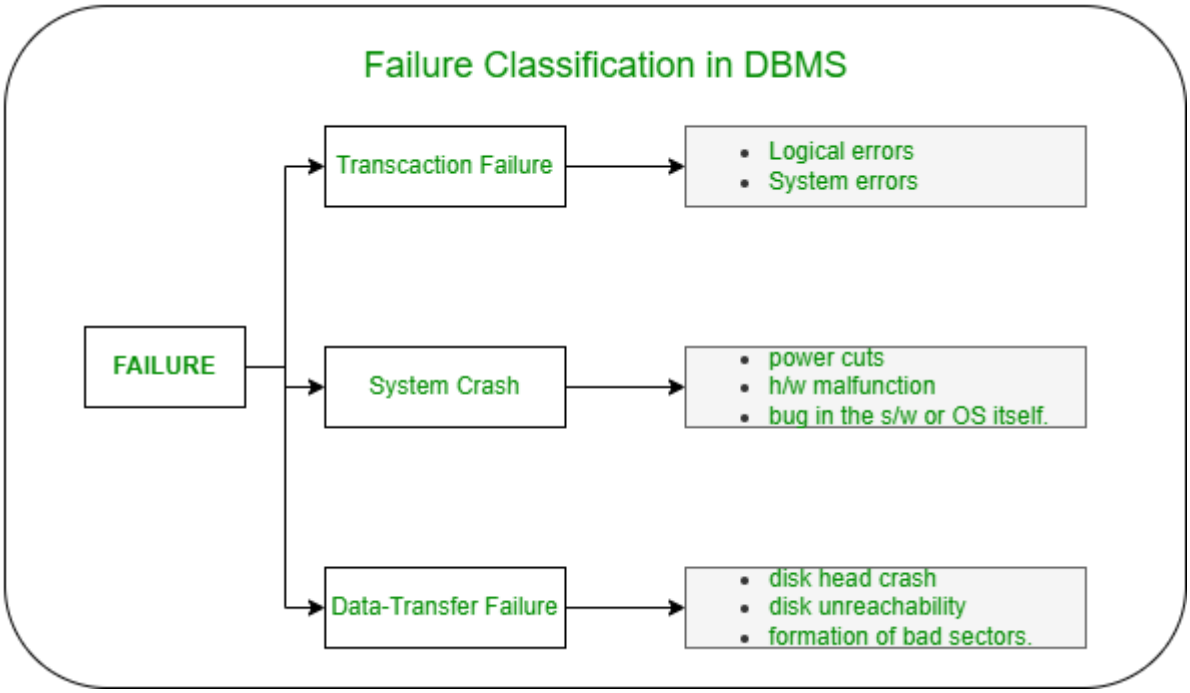
There is such a thing as perfect isolation (we will define this below). Unfortunately, perfection usually comes at a performance cost—in terms of transaction latency (how long before a transaction completes) or throughput (how many transactions per second can the system complete). Depending on how a particular system is architected, perfect isolation becomes easier or harder to achieve. In poorly designed systems, achieving perfection comes with a prohibitive performance cost, and users of such systems will be pushed to accept guarantees significantly short of perfection. However, even in well-designed systems, there is often a non-trivial performance benefit achieved by accepting guarantees short of perfection. Therefore, isolation levels came into existence: they provide the user of a system the ability to trade off isolation guarantees for improved performance.

FAILURE CLASSIFICATION IN DBMS

Failure in terms of a database can be defined as its inability to execute the specified transaction or loss of data from the database. A DBMS is vulnerable to several kinds of failures and each of these failures needs to be managed differently. There are many reasons that can cause database failures such as network failure, system crash, natural disasters, carelessness, sabotage(corrupting the data intentionally), software errors, etc.

Failure Classification in DBMS

A failure in DBMS can be classified as:



Transaction Failure:

If a transaction is not able to execute or it comes to a point from where the transaction becomes incapable of executing further then it is termed as a failure in a transaction.

Reason for a transaction failure in DBMS:

Logical error: A logical error occurs if a transaction is unable to execute because of some mistakes in the code or due to the presence of some internal faults.

System error: Where the termination of an active transaction is done by the database system itself due to some system issue or because the database management system is unable to proceed with the transaction. For example– The system ends an operating transaction if it reaches a deadlock condition or if there is an unavailability of resources.

System Crash:

A system crash usually occurs when there is some sort of hardware or software breakdown. Some other problems which are external to the system and cause the system to abruptly stop or eventually crash include failure of the transaction, operating system errors, power cuts, main memory crash, etc.

These types of failures are often termed soft failures and are responsible for the data losses in the volatile memory. It is assumed that a system crash does not have any effect on the data stored in the non-volatile storage and this is known as the fail-stop assumption.

Data-transfer Failure:

When a disk failure occurs amid data-transfer operation resulting in loss of content from disk storage then such failures are categorized as data-transfer failures. Some other reason for disk failures includes disk head crash, disk unreachability, formation of bad sectors, read-write errors on the disk, etc.

DATABASE STORAGE STRUCTURE

All the information in a database is organized and structured in database tables. These tables are stored on the hard disk of the database server. The database tables are usually divided into columns and rows, just like a regular graphic table. In a database table, the columns specify the information category and the data type and the rows hold the actual information. This structure is chosen for its ease of use – it can be easily indexed, accessed or modified.

Control Files

A control file tracks the physical components of the database and other control information. It is essential to the functioning of the database. Because of the importance of the control file, Oracle recommends that the control file be multiplexed. In other words, the control file should have multiple identical copies. For databases created with DBCA, three copies of the control file are automatically created.

Tablespaces

A database consists of one or more tablespaces. A tablespace is a logical structure created by and known only to the Oracle database server. A tablespace consists of one or more datafiles or tempfiles. There are various types of tablespaces, including the following:

- Undo tablespace
- Permanent tablespaces
- Temporary tablespaces

Datafiles

Datafiles are the operating system files that hold the data within the database. The data is written to these files in an Oracle proprietary format that cannot be read by programs other than the database server. Tempfiles are a special class of datafiles that are associated only with temporary tablespaces.

Redo Log FIle

This physical file is charged with storing the redo log buffer content. Because of the nature of its content, it is multiplexed. This file is stored in a redo log group and each group must have at least one redo-log file. A database must have at least two redo-log groups. The LGWR process is charged with writing the log buffer content to the active log file.

Parameter File

This file stores information about the initialisation parameters (settings) used when the instance is starting. Oracle uses this file's information to determine how to start the instance. This file can be a plain file i.e. P file or a binary file ie. SP file. You can use either files to configure both instance and database options. SP file is the most recommended file for database startup due to its dynamic nature.

Alert Log File

This is an XML file also known as alert.log charged with storing a chronological log of messages and faults written out by the database. Database startup, shutdown, log switches, space issues, and other alerts are common in this file. This file should be checked on a regular basis for any unusual messages or corruptions. When the old alert log file is destroyed, Oracle will immediately create a new one. To view this file, we can use the code below.

Other Storage Structures

Other storage structures that can exist in an Oracle database include the initialization parameter file, the password file, and backup files.

Initialization Parameter File

Initialization parameters are used by the database server at startup to determine the runtime resources for the database. They are actively monitored by the database and can be set or modified while the database is running.

Password File

A database can use a password file to authenticate administrative users with SYSDBA connect privileges. SYSDBA privileges enable a DBA to start up and shut down the database and perform other high-level administrative tasks. This password file is outside of the database itself because it must sometimes be referenced when the database is not yet running.

This is not the only form of administrator authentication, so not all databases require a password file.

Backup Files

Backup files are technically not database files, but rather copies of the database in some form that can be used to recover the database if a failure causes loss of data.

RECOVERY AND SHADOW PAGING

Shadow Paging is recovery technique that is used to recover database. In this recovery technique, database is considered as made up of fixed size of logical units of storage which are referred as pages. pages are mapped into physical blocks of storage, with help of the page table which allow one entry for each logical page of database. This method uses two page tables named current page table and shadow page table.

RECOVERY WITH CONCURRENT TRANSACTION

Whenever more than one transaction is being executed, then the interleaved of logs occur. During recovery, it would become difficult for the recovery system to backtrack all logs and then start recovering.

To ease this situation, 'checkpoint' concept is used by most DBMS.

BUFFER MANAGEMENT

A Buffer Manager is responsible for allocating space to the buffer in order to store data into the buffer. If a user request a particular block and the block is available in the buffer, the buffer manager provides the block address in the main memory.

BUFFER POOL

A buffer pool is an area of main memory that has been allocated by the database manager for the purpose of caching table and index data as it is read from disk.

FRAME REPLACEMENT POLICY

Buffer replacement is a method of replacing existing blocks in the buffer with the new block. By doing so, it helps to reduce the number of disk access. The buffer manager uses the following strategies for Buffer Replacements: Least Recently Used (LRU) Strategy, Toss-immediate Strategy, Most Recently Used (MRU) Strategy

BUFFER ALLOCATION

The buffer allocation model of NvSciBuf is summarized as follows: If two or more hardware engines wants to access a common buffer (e.g., one engine is writing data into the buffer and the other engine is reading from the buffer)